ENCE260: Embedded Systems

Lecture 6: Concurrency

Computers can multitask, right?

Revision

For our 5×7 display, what is the smallest number of time-division-multiplexing states that will allow all possible LED patterns?

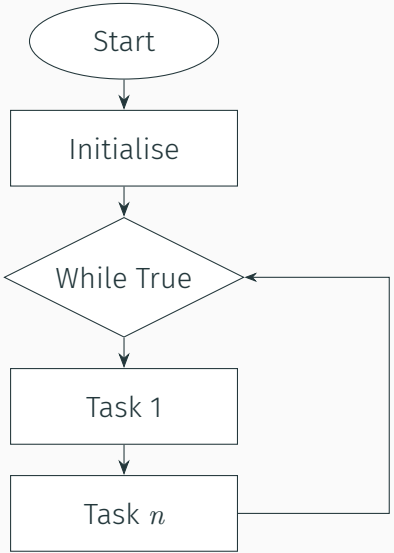
- 35
- 12
- 7
- 5 ✓

Concurrency

- Concurrency is when two or more parts of a program will conceptually run at the same time.
- A parallel program is concurrent, but concurrent programs don't have to be parallel; we can multiplex here, too!
- We can also think of concurrency as multitasking, from a user perspective.
- For our game on the UCFK4, we need to poll buttons, refresh the display, update the game status etc., concurrently. (We only have 1 CPU!)
- We use a *scheduler* program to share the CPU.

Cyclic Executive / Round Robin

- **Pros:**
  - Easy to implement.
  - Response times are the same for every task (equal frequencies).
  - Good for background tasks.
- **Cons:**
  - Not all tasks have the same priority in most applications.
  - A long-running task can hog the CPU.



Delay Loops

```
1  int main(void)
2  {
3      while (true) {
4          doThing1();
5
6          doThing2();
7
8          delay_ms(LOOP_PERIOD);
9      }
10 }
```

- This loop won't reliably take `LOOP_PERIOD` to execute.
- The time taken to do the tasks, as well as the loop overhead, will make loop times inconsistent and longer than desired.
- We therefore need a better option than just putting a delay inside a loop.

Paced Loops

- A paced loop allows us to synchronise tasks.
- The Timer-Counter register (here `TCNT1`) is reset *after* the `while` loop.
- This is equivalent to zeroing the counter before the tasks, and putting the loop after the tasks.

```
1  void pacer_wait(void)
2  {
3      while (TCNT1 <
       period_ticks) {
4          continue;
5      }
6
7      TCNT1 = 0;
8  }
9
10 int main(void)
11 {
12     while (true) {
13         doThing1();
14         doThing2();
15
16         pacer_wait();
17     }
18 }
```

Calculation of Ticks in C

- To calculate the number of ticks from the desired loop frequency:

$$\text{ticks} = \frac{T}{\Delta t}. \qquad T = \text{delay} = \frac{1}{\text{frequency}}.$$

$$\text{resolution} = \Delta t = \frac{\text{prescaler}}{f_{\text{clk}}}. \qquad \text{ticks} = \frac{f_{\text{clk}}}{\text{frequency} \times \text{prescaler}}.$$

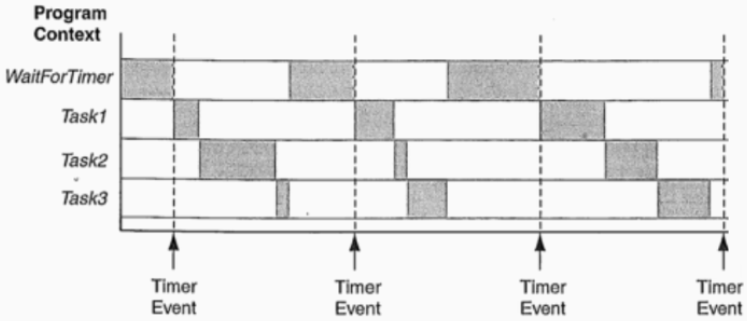
```
#define CPU_F 8000000
#define PRESCALER 1024

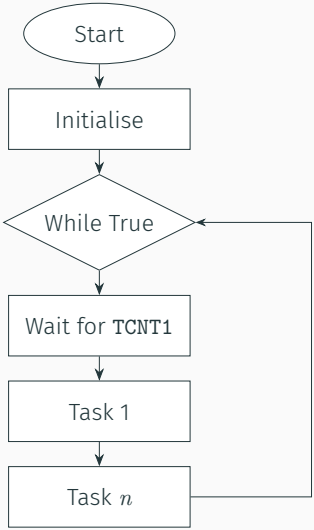
static uint16_t period_ticks;

void pacer_init(uint16_t frequency)
{
    period_ticks = (CPU_F/PRESCALER)/frequency;
}
```

Paced Loop Round Robin Scheduler

- `pacer_wait()` uses `TCNT1` to update itself.
- The net task duration cannot exceed the pacer period, otherwise there will be timing jitter.



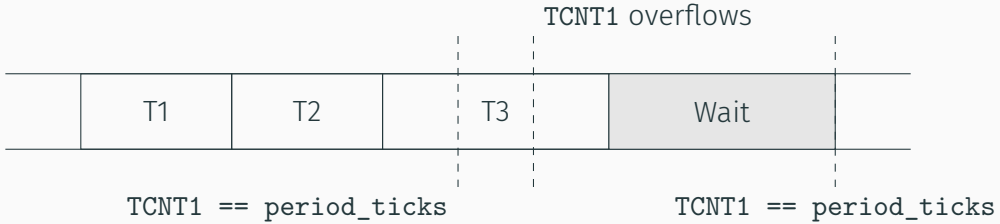


Paced Round Robin Example

```
1  // Provided library file.
2  #include "pacer.h"
3
4  // Other includes...
5
6  void main(void)
7  {
8      pacer_init(200); // Sets pacer period to correct value for a 200 Hz
       loop.
9
10     // Initialisation for tasks should go here.
11
12     while (true) {
13         // Example tasks, all with a period of 5 ms.
14         display_refresh();
15         button_poll();
16         game_update();
17
18         pacer_wait(); // Paces the loop.
19     }
20 }
```

Problems With Paced Loop Round Robin

- When a task runs over its allocated time, all other tasks will be delayed, meaning they fail to stay within deadlines.
- Any task that overruns will occupy the CPU until it finishes.
- If `TCNT1` overflows during a task, this is even worse:



- Tasks may get called more often than required, since all run at the same rate. We can overcome this with selective task execution.

Paced Loop with Different Task Periods

```c
#include "pacer.h"

void main(void)
{
    uint8_t buttonTicks = 0; // Extra value to change task rate.
    pacer_init(200); // Sets pacer period to correct value for a 200 Hz
    loop.

    // Initialisation for tasks here.

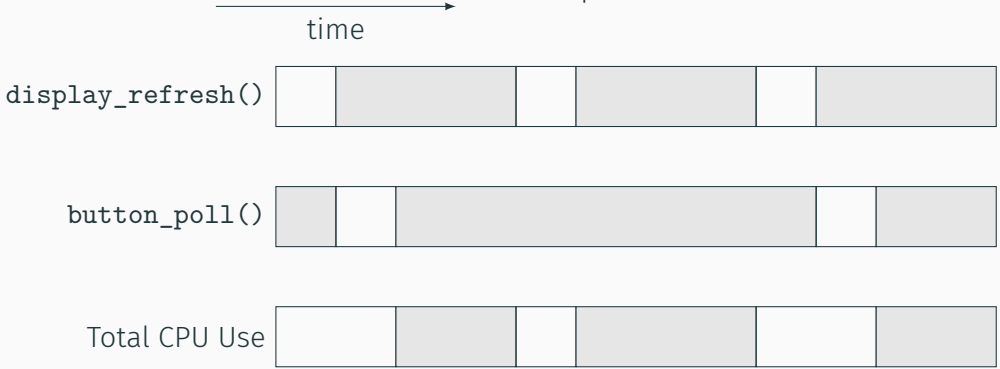
    while (true) {
        display_refresh(); // This task runs at 200 Hz, 5 ms period.

        buttonTicks++;
        if (buttonTicks >= 20) { // Only run once every 20 loops.
            buttonTicks = 0; // Reset the counter.
            button_poll(); // Runs at an effective 10 Hz, 100 ms period.
        }

        pacer_wait(); // Paces the loop.
    }
}
```

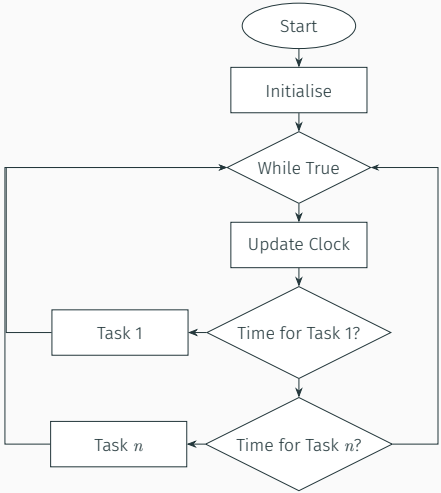
Periodic Task Scheduling

- In periodic task scheduling, timer ticks are used to schedule the running of individual tasks.
- This allows tasks to run at different frequencies.



Periodic Task Scheduler with Priority

- We don't hog the CPU like with the pacer loop.
- We cannot interrupt a task to perform a higher priority task (interrupts ENCE361, preemption ENCE360).
- Timing jitter can still occur if a task takes too long to complete.
- A task can hog the CPU meaning other tasks don't run.



Periodic Scheduler Fun Kit Library

```c
// Library containing a scheduler. If you're comfortable with function pointers, you're free to use this for your game.
#include "task.h"

// Here, tasks take no arguments and return nothing.
static void task1 (void *data)
{
    // Do Task
}

// In this example, tasks are static: only visible to this module.
static void task2 (void *data)
{
    Task2State state = *data; // Extract passed param for task if desired.
}

int main (void)
{
    task_t tasks[] = { // Array of task structs with configuration.
        {.func = &task1, .period = TASK_RATE / 2},
        {.func = &task2, .period = TASK_RATE / 3},
    };

    task_schedule(tasks, 2); // Contains an internal `while (true)`.
}
```

CPU Load

- A CPU can only run so many tasks before it simply can't keep up.
- For a pacer, this happens when there is no time for a delay at the end of the loop (the loop has run over).
- For a periodic scheduler, tasks are waiting around "ready" without being run.
- To quantify this, we calculate the CPU load for a set of N tasks:

$$\text{Load} \quad = \quad \frac{1}{T_p} \sum_{i=1}^{N} T_i \quad = \quad \frac{\text{Sum of execution times}}{\text{Execution period}}.$$

- Can determine with equation once tasks repeat, or experimentally.

Key Concepts

- Even with a single CPU, we can run multiple tasks “at the same time” by timesharing.
- The way we interleave tasks is determined by our *scheduler*.
- To maintain timing, any waiting in the main loop should be done using a *pacer*, not a constant delay.
- Scheduler options include simple round-robin, periodic round-robin, and periodic with priority.
- Regardless of scheduler, a given microcontroller can only handle so many tasks before becoming overloaded.

Homework

An embedded system using round-robin is running:

- **Task 1:** A 20 Hz task that takes 20 ms;
- **Task 2:** A 100 Hz task that takes 3 ms.

What is the CPU load?

Does changing the scheduler used change your answer?